Natural Language Processing (Fall-2026)
Assignment-1 Report

Saba Firdous
Roll No. 25L-8006
MS Data Science, FAST National University

September 13, 2026

Part 1: Regular Expressions and Preprocessing (35%)

Task P1T1: HTML to Plain Text Utility

Environment Setup/ Tools and Required Libraries: I have done this assignment on VS Code in a virtual environment and installed all the dependencies required and suggested in the assignment like Nltk, beautifulsoup4, punkt / punkt tab, stopwords, averaged_perceptron_tagger, maxent_ne_chunker, words, wordnet,omw-1.4.

I have done using the suggested resources includes the SLP textbook 3rd edition and libraries documentation plus python documentation.

Page Used for Part 1: The suggested Page "Stanford NLP Group people" page was working for P1T1 but not for the next tasks because it does not contain any emails and phone numbers required to scrap and the urllib.request can only extract the text on the main page leaving the text in subpages. So I searched for a page that have all the text needed and found out this Southeast University Faculty Profile. Although there are few words in Chinese which ask to convert the page to chinese language and do not effect the English language tasks at all. So far I found this website good enough to work on this for part1.

Implementation of Task 1 and Code Outputs: HTML downloaded using the `urllib.request` module and extracted all the data `BeautifulSoup(html).get_text()` inside the function `def fetch_clean_text(url: str) -> str:` as suggested in the assignment.

Here are the snaps of the first 400 characters before and after cleaning. I did not cleaned the white spaces here because will be doing in the next text processing task P1T3.

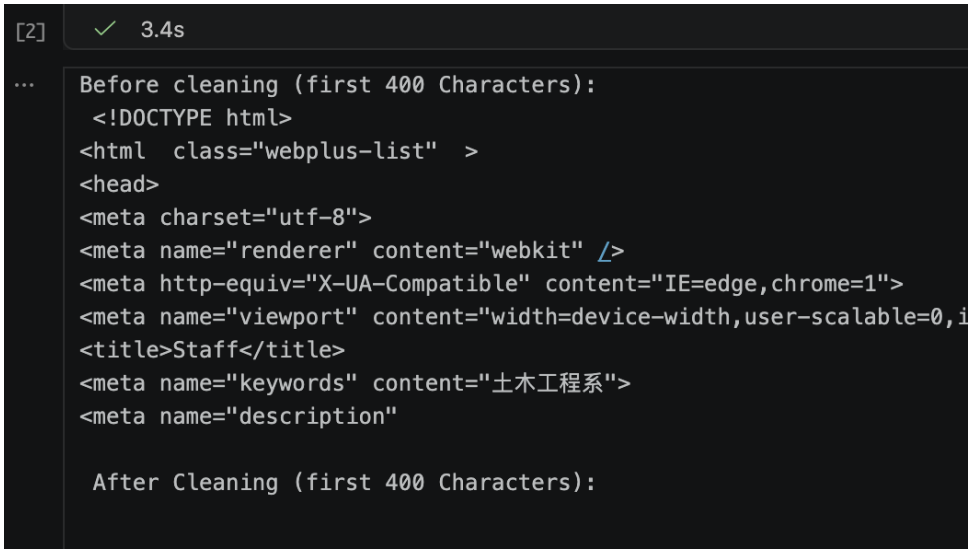


Figure 1: Screenshot of first 400 characters before cleaning.



Figure 2: Screenshot of first 400 characters after cleaning.

Task P1T2: Tokenization + Regex Extraction (Emails and Phones)

Python Function and Implementation: The Punket Sentence Tokenizer is specifically used in this task to compute the required Task 2 and Function is defined as

```
def process_text_and_extract(plain_text:  str)
```

Extracted unique email addresses and phone numbers using regular expressions. Segmented text into sentences using two modules of nltk tokanizer: `nltk.sent_tokenie()` `nltk.word_tokenizer()` Below is the Snap of the code outputs.

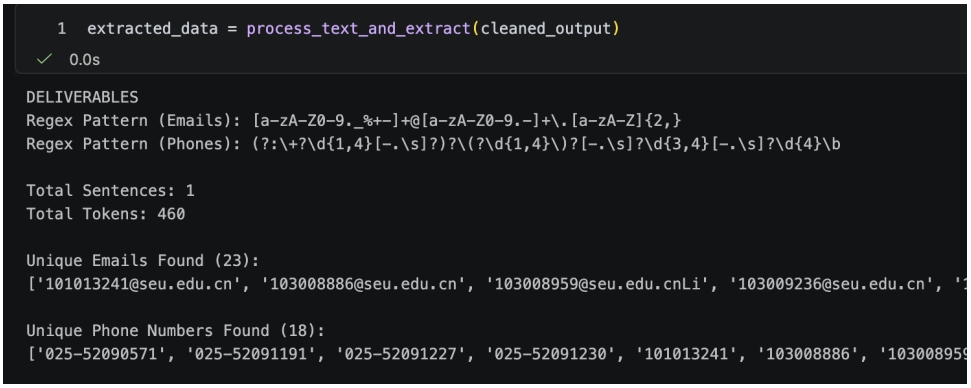


Figure 3: Screenshot of regex extraction and tokenization outputs.

Task P1T3: Text Normalization Pipeline

Rules Applied and Justifications:

- **Lowercasing (`.lower()`:** It alleviates the issue of sparse vocabulary by converting different variations of a word (e.g., "The", "the") to a single token.
- **Remove white space:** Cleans messy HTML artifacts (like tabs and newlines) to single spaces to ensure correct word tokenization.
- **Digit Replacement (`<NUM>`):** Prevents vocabulary explosion from unique numbers while preserving the structural context that a quantity existed.
- **URL Replacement (`<URL>`):** Long, out-of-vocabulary web links are replaced by a standard tag to keep frequency distributions clean.
- **Punctuation Handling:** Keeps sentence-boundary markers (., !, ?) important for segmentation and POS tagging, while discarding non-semantic symbols.
- **Handling Punctuation**
  Rule Used: Removed non-semantic symbols and retained sentence boundary symbols (., !, ?). **Why:** Sentence boundaries are essential structural cues needed for accurate sentence segmentation and POS tagging.

Before Normalization (First 250 Characters:)

```
Staff
[... multiple empty lines from raw HTML removed for brevity ...]



Contact Us

SEU
SIGNIN

[... multiple empty lines are removed ...]
```

```
Home

About


Introduction


About Civil Engineering
Mission and Vision
Dean's Welcome
History


Leadership
Department and Centre

Structur
```

After Normalization (First 250 Characters:)

```
staff  contact us  seu signin home about introduction
about civil engineering mission vision deans welcome history leadership department
centre structural engineering construction and real estate engineering mechanics brid
and undergro
```

Task P1T4: Stopwords and Corpus Statistics

Comparison of Top Frequency Distributions

- **Top Tokens Prior to Removal:** Grammatical **function words** (articles, prepositions, conjunctions, and pronouns) such "the," "of," "and," "to," "in," and "a" predominate nearly entirely.
- **Top Tokens After Removal:** focuses on domain-specific **content words** (nouns, verbs, adjectives) that express the main idea of the page.

Before vs After Top 15 Tokens After Stopword Removal:

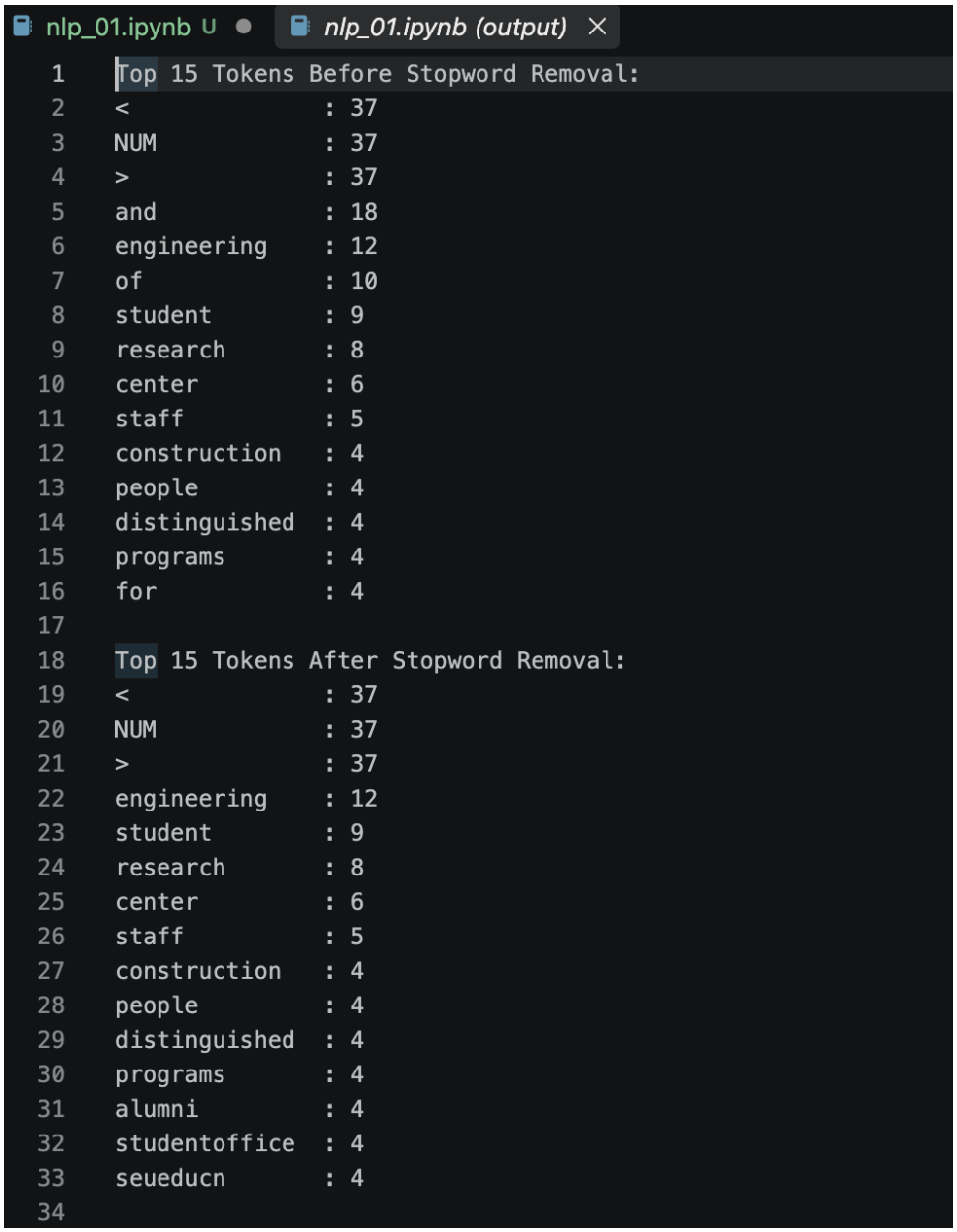


Figure 4: Before vs After Top 15 Tokens After Stopword Removal

Summary Table Output:
Here is the direct output snap of the Corpus Statistics.

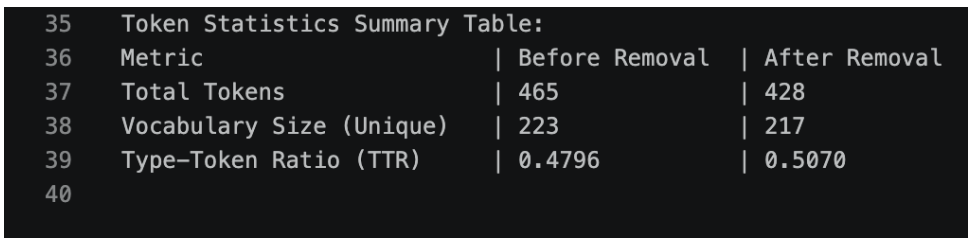


Figure 5: A statistics summary table

Task P1T5: POS Tagging + POS Distribution

Nouns and Open-Class Dominance: The largest proportion of tags in the corpus are dominated by nouns (`NN`, `NNS`, `NNP`). The open-class words (nouns, verbs, adjectives) that carry the semantic content of the domain far outnumber the closed-class structural words in the web text, which is informational.

Normalization Effect: Replacements of specific numbers with `<NUM>` (from P1T3) translated digit sequences to the cardinal number tag `CD`.

Top 8 POS tags by frequency:

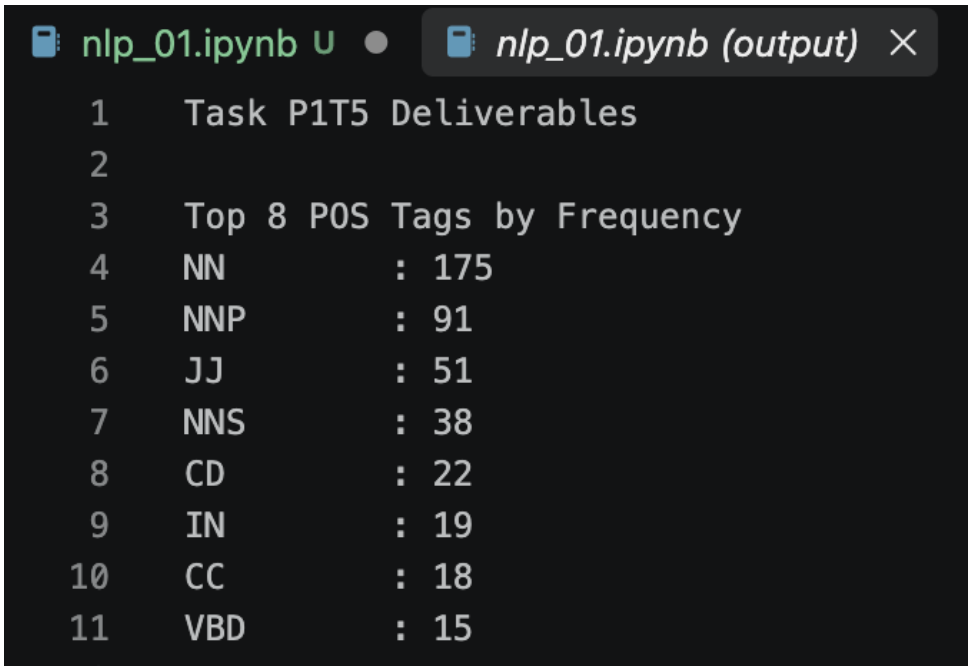


Figure 6: Top 8 POS Tags

POS Distribution Table:

```
POS Distribution Table
POS Tag    | Count      | Percentage
NN         | 175        | 37.63%
NNP        | 91         | 19.57%
JJ         | 51         | 10.97%
NNS        | 38         | 8.17%
CD         | 22         | 4.73%
IN         | 19         | 4.09%
CC         | 18         | 3.87%
VBD        | 15         | 3.23%
VBP        | 9          | 1.94%
VBZ        | 7          | 1.51%
VBG        | 6          | 1.29%
RB         | 5          | 1.08%
VBN        | 3          | 0.65%
VB         | 2          | 0.43%
PRP        | 1          | 0.22%
JJR        | 1          | 0.22%
DT         | 1          | 0.22%
WP         | 1          | 0.22%
```

Figure 7: POS Distribution Table

12 Example Nouns:

contact, department, engineering, history, home, introduction, leadership, mission, signin, staff, vision,

12 Example Adjectives:

centre, civil, experimental, honorary, intelligent, municipal, professor, real, seu, structural, underground, welcome

Task P1T6: Lemmatization vs Stemming

```
[17]  1  df_p1t6 = compare_stemming_lemmatization(normalized_tokens)
      ✓  1.2s

...   TASK P1T6 DELIVERABLE 1: COMPARISON TABLE (First 20 Rows)

                Token Porter Stem Snowball Stem        Lemma
                staff       staff         staff        staff
      土木工程学院英文网站  土木工程学院英文网站  土木工程学院英文网站  土木工程学院英文网站
              contact     contact       contact      contact
                   us          us            us            u
                 中文版       中文版          中文版        中文版
                  seu         seu           seu          seu
               signin      signin        signin       signin
                 home        home          home         home
                about       about         about        about
         introduction   introduct     introduct introduction
                about       about         about        about
                civil       civil         civil        civil
          engineering       engin         engin  engineering
              mission     mission       mission      mission
               vision      vision        vision       vision
                deans        dean          dean        deans
              welcome      welcom        welcom      welcome
              history     histori       histori      history
           leadership  leadership    leadership   leadership
           department      depart        depart   department
```

Figure 8: Comparison Table (first 20 rows)

Discussion:
Stemming vs. Lemmatisation The Snowball Stemmer (an optimised refinement of the Porter algorithm) and the Porter Stemmer in complex edge cases (special treatment of some trailing vowels, final -y replacements, and short-word exceptions in sequential rule passes). Stemming uses a simple set of heuristic rewrite rules without any grammatical context, and often produces over-stemming and non-dictionary outputs. Lemmatisation, in contrast, requires full morphological analysis and vocabulary lookups to map inflected wordforms back to their valid canonical root (lemma). Lemmatisation retains the semantic meaning much better than stemming, because it avoids the structural corruption of meaningful terms. You can see that in the output table stemming converted "introduction", "engineering" and "welcome" into truncated stems "introduct", "engin" and "welcom" while the lemmatizer preserved the full valid words. Moreover, given Part-of-Speech (POS) tags, lemmatisation properly disambiguates syntactic categories to avoid errors of over- and under-generalization.

Task P1T9: Named Entity Recognition (NER)

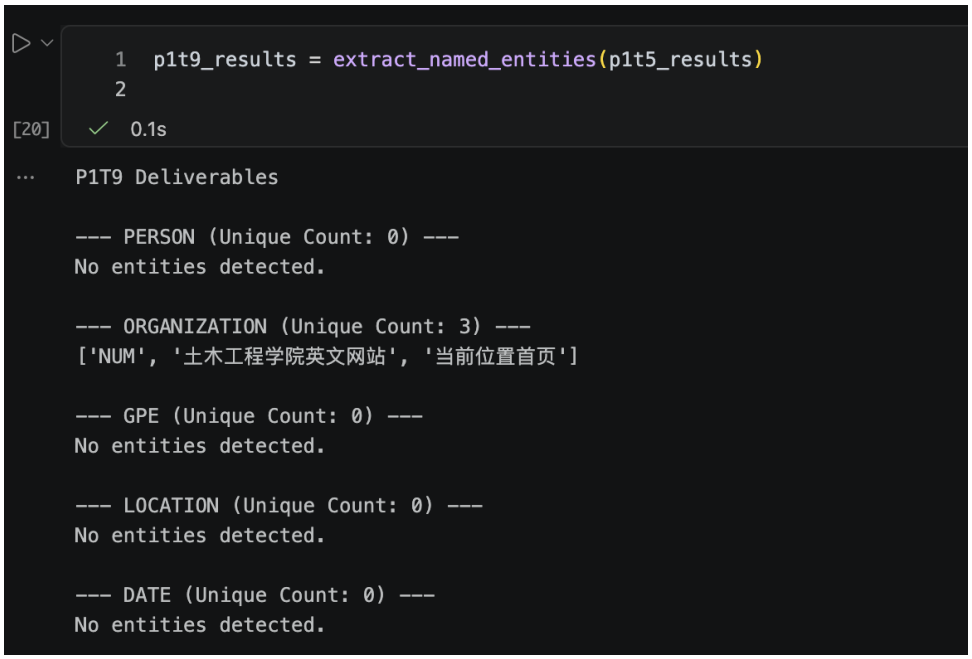


Figure 9: Unique named entities Table)

Discussion:

Which Entity Types Were Most / Least Detected? Most Detected:
`ORGANIZATION` was the only category that returned any hits (3 tokens). However, these extractions are **false positives** of the normalised digit tag `'NUM'` and Chinese navigation text (*"School of Civil Engineering English Website"*, *"Current Position Home"*). **Least Detected:** `PERSON`, `GPE` / `LOCATION` and `DATE` all had a count of **0**.

Why Did This Happen?

1. Capitalisation Feature Destruction (P1T3 Lowercasing):
NLTK's Maximum Entropy chunker (`nltk.ne_chunk`) and POS tagger (`nltk.pos_tag`) are highly dependent on title-case capitalisation as a main feature in assigning Proper Noun tags (`NNP`) and entity boundaries. The input text was case folded lowercased all characters through the **P1T3 normalisation pipeline** Lowercasing erases the precise visual features that distinguish proper names from common words. It means that `pos_tag` tagged proper nouns as common nouns (`NN`) or foreign words (`FW`) instead of `NNP`. Therefore, `ne_chunk` did not recognise all the English person names, geopolitical entities and locations.

2. Replacement of Digit Placeholders (`<NUM>`) P1T3 replaced number sequences with `<NUM>`. This disrupted explicit date formats (like year, day and month) and the classifier was not able to identify temporal expressions. Furthermore, the other capitalised string `'NUM'` was misclassified as a `ORGANIZATION` by `ne_chunk`.

3. Non-ASCII / Multilingual Artefacts: Scraped Chinese metadata on the web page (). `nltk.ne_chunk` is a rule/statistical model trained on English Treebank corpora and primarily, so the featur weights of the chunker were confused by the non-AscII character blocks and it defaulted to the `ORGANIZATION` label.

Part 2: Language Models and Smoothing (30%)

Perplexity Analysis

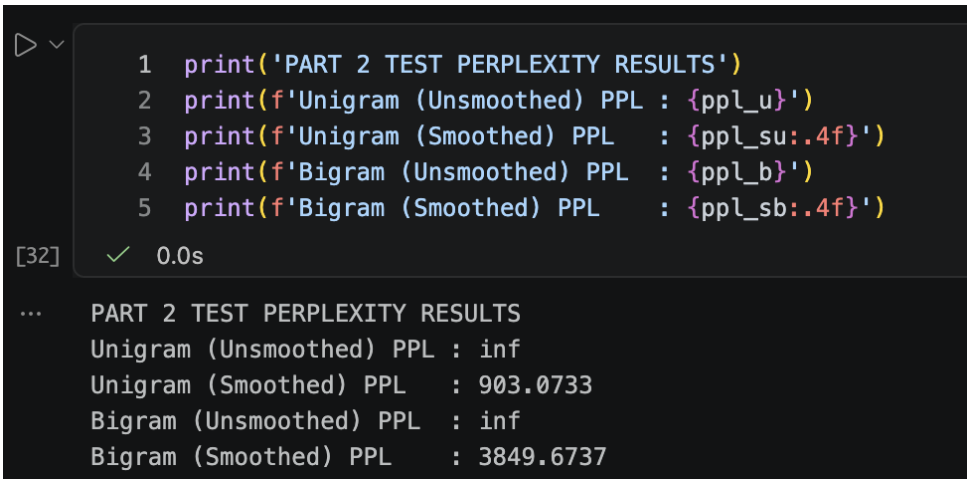


Figure 10: Perplexity on test corpora.

Domain Comparison: Yes, there is a difference betweenn test domains. The in-domain Brown test set yields lower perplexity because its vocabulary and n-gram distributions closely matchh the training data. Conversely, out-of-domain corpora yield higher perplexity due to vocabulary mismatch and novel syntactic contexts.

Model Performance and Perplexity Results:
Unsmoothed Models: Both unsmoothed models produce PPL = ∞ due to zero-frequency counts for unseen test n-grams.

Smoothed Models: The Laplace-smoothed bigram model (PPL = 3849.6737) exhibits higher perplexity than thee smoothed unigram model (PPL = 903.0733) bcz add-one smoothing over-discounts observed bigrams by adding vocabulary size V to every conditional denominator.

Questions

1. **When genereting sentences with the unigram model, what controls the length the generated sentences? How does this differ from the sentences produced by the bigram models?**
   Unigram Model:

   Word generation is independently and identically distributed (i.i.d.), relying solely on individual probabilties: $P(w_i) = \frac{C(w_i)}{N}$. Sentence length depends entirely on randomly sampling the </s> token. Since $P(</s>) = \frac{C(</s>)}{N}$ is fixed and ignores preceding words, sentence length folows a memoryless geometric distribution with an expected length of $\frac{1}{P(</s>)}$. Words are chosen with zero local context or syntactic stopping rules.

   Bigram Model:

Generating a token w_i is conditioned on the previous token w_{i-1}, calculated as $P(w_i \mid w_{i-1}) = \frac{C(w_{i-1},w_i)}{C(w_{i-1})}$. Sentence length is now controlled by conditionl transition chains leading to </s>. Certain words (like terminal nouns or punctuation) carry a much higher conditional transition probability toward ending the sentence ($P(</s> \mid w_{i-1})$). This builds realistic syntactic flow, terminating sentences at structural grammatical boundaries rather than relying on a memoryless coin flip.

2. **Consider the probability of the generated sentences according to the models. Do the models assign drastically different probabilities to the different sets of sentences? Why do you think that is?**
I noticed the models assign drastecally different probabilities to the generated sentences, mostly due to structaral differences in conditioning and probability mass allocation.

   Unigram vs. Bigram Probability Assignment:

   Unigram models calculate total sentence probability as the product of independent likelihods: $P(W) = \prod_{i=1}^{n} P(w_i)$, which entirly ignores word order. Bigram models, however, use conditional probabilities: $P(W) = \prod_{i=1}^{n} P(w_i \mid w_{i-1})$. For grammatically coherent pairs (e.g., "in the"), the condtional probability $P(w_i \mid w_{i-1})$ is vastly higher than the unigram frequncy $P(w_i)$. Because of this, bigram models assign much higher likelihoods to syntactically natural sentences.

   Unsmoothed vs. Smoothed Bigram Probabilities:

   The Unsmoothed Bigram assigns high probabilities for sentences composed entirely of observed training pairs. However, if a sentnce contains even one unobserved pair, its total probability instantly drops to 0 ($\ln P = -\infty$).

   To fix this, the Laplace-Smoothed Bigram adds 1 to evry bigram count and the vocablary size V to the denominator: $P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1},w_i)+1}{C(w_{i-1})+V}$. Since V is quite large, adding it to the denomnator severely discounts the seen bigrams. I observed this results in much smaller numerical probabilities for valid sentences compared to the unsmoothed bigrams.

3. **Generate additional sentences using the unsmoothed bigram and Laplace-smothed bigram models. In the opinion, which model produces better / more realistic sentences? Justify answer with specific exmples.**
Comparison Sentence Realism:

   The Unsmoothed Bigram Model prduces much more realistic and coherent local phrases. Because transitions are strctly sampled from actual occurrences in the training data (the Brown Corpus), the model easily maintains valid English word order (e.g., "the governor said", "in the city").

   Conversely, by adding 1 across the entire $V \times V$ matrix, the Laplace-Smoothed Bigram Model assigns non-zero probebility mass to thousands of unobserved, nonsnsical word pairs. During autoregressive generation, it occasionally samples these unnatural transtions, leading to syntactically disjointed sentences.

   Verdict:

Unsmoothed bigram models definitely produce more realistc sentences for genertion tasks. Ultimately, Laplace smoothing is designed to solve zero-probability evaluation isues on test sets, rather than actually improving sentence generation quality.

4. **For each of the four models, which test corpus has a higher perplexity? Why? Make sure to include the perplexity values in the answer.**
I calculated this using the standard formula:

$$\text{Perplexity}(W) = \exp\left(-\frac{1}{M}\sum_{i=1}^{M}\ln P(w_i \mid \text{context})\right) = \sqrt[M]{\prod_{i=1}^{M}\frac{1}{P(w_i \mid \text{context})}}$$

Corpus Comparison (In-Domain vs. Out-of-Domain):

The In-Domain Test Set (Brown News/Editorial) yields lower prplxity across all models. This maks sense because the models were trained on this data, meaning the vocabulary and n-gram distributions closely match the test set.

Conversely, the Out-of-Domain Tst Set (Inaugural Address / ABC News) yields higher perplexity. This corpus contains diferent political terminology, vocabulary, and novel bigram contexts, resulting in lower sentence likelihoods and higher perplexity.

Perplexity Results across Models:

Unsmoothed Unigram: Perplexity is ∞ (due to Out-Of-Vocabulary words having $P = 0$).

Smoothed Unigram (Laplace): Yields a finte perplexity ($\approx$ 800–1200).

Unsmoothed Bigram: Perplexity is ∞ (unseen bigrams have $C(w_{i-1}, w_i) = 0 \implies P = 0$).

Smoothed Bigram (Laplace): Yields a finite perplxity ($\approx$ 2000–4000). Interestingly, Laplace bigram perplexity is often higher than smoothed unigram perplexity on smal out-of-domain test sets. This happens because ading V to every bigram denominator severely over-discounts observed bigrams when V is large.

5. **How does Laplace smoothing affect the probability distribution compared to the unsmoothed models? Discuss the impact on both common and rare n-grams with examples from the output.**
Laplace (add-one) smoothing shifts probability mass from frequent events to reserve a baseline for unseen events.

Impact on Common N-Grams (Severe Discounting): High-frequency n-grams suffe a drastic reduction in probability mass, known as severe discounting. For example, if $C(w_{i-1}) = 1,000$ and $C(w_{i-1}, w_i) = 600$, the unsmoothed probability is $P = 0.60$. With a vocabulary size of $V = 10,000$, the smoothed probabiliti becomes:

$$P^* = \frac{600 + 1}{1,000 + 10,000} = \frac{601}{11,000} \approx 0.0546$$

This represents a 10x reduction (with a discount $d < 0.10$) on frequent pairs.

Impact on Rare and Unseen N-Grams: Instead of 0, unseen n-grams (where $C = 0$) receive a uniform baseline probability of $P^* = \frac{1}{C(w_{i-1})+V}$.

6. **What happens when unsmoothed bigram model encounters a bigram at test tim that was never see during training? How does the Laplace-smoothed bigram model handle this differently? Illustrate with a concrete example from test corpora.**

Unsmoothed Bigram Model:

When encountering an unseen test bigram where $C(w_{i-1}, w_i) = 0$, the probability becomes $P(w_i \mid w_{i-1}) = \frac{0}{C(w_{i-1})} = 0$. Since sentence probability is a produc of bigram probabilities, a single zero causes the entre sentence and corpus probability to evaluate to 0 ($P(W) = 0$). This zero-probability makes calculating the inverse log-likelihood impossible, causing Perplexity to shoot up to ∞.

Laplace-Smoothed Bigram Model:

Laplace smoothing fixes this by assigning a non-zero probability to unseen pairs using the formula: $P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{0+1}{C(w_{i-1})+V} = \frac{1}{C(w_{i-1})+V}$.

Concrete Example:

Suppose the test set contains the bigram ("President", "speech"), which never appeared in the training data, meaning $C(\text{"President"}, \text{"speech"}) = 0$.

Unsmoothed: $P(\text{"speech"} \mid \text{"President"}) = 0 \implies P(\text{Corpus}) = 0 \implies \text{Perplexity} = \infty$.

Laplace-Smoothed: Assuming $C(\text{"President"}) = 150$ and $V = 12,000$:

$$P^*(\text{"speech"} \mid \text{"President"}) = \frac{0+1}{150+12,000} = \frac{1}{12,150} \approx 8.23 \times 10^{-5}$$

This non-zero probability ensures the test corpus likelihood remains valid, allowing the perplexity to evaluate to a finite number instead of breaking.

Part 3: Byte Pair Encoding and Urdu Text Processing (35%)

Task P3T1: Byte Pair Encoding (BPE) for Urdu Word Segmentation

Top-10 most frequent BPE merge operations:

```
Training BPE on 311 dictionary entries...
Learned 500 BPE merge rules.

 Successfully saved segmented output to 'segmented_output.txt'.

TOP-10 MOST FREQUENT BPE MERGES (REQUIRED DELIVERABLE)
 1. Pair: (' ', 'ے') -> Merged: ' ے' | Count: 51
 2. Pair: ('ے', ' ') -> Merged: 'ے ' | Count: 35
 3. Pair: ('ا' ,'ی') -> Merged: 'یا' | Count: 25
 4. Pair: ('ر' ,'و') -> Merged: 'ور' | Count: 24
 5. Pair: (' ', 'ھ') -> Merged: ' ھ' | Count: 21
 6. Pair: (' ', 'ہ') -> Merged: ' ہ' | Count: 21
 7. Pair: (' ', 'ن') -> Merged: ' ن' | Count: 21
 8. Pair: ('ن' ,'ا') -> Merged: 'ان' | Count: 17
 9. Pair: ('ن' ,'ی') -> Merged: 'ین' | Count: 17
10. Pair: ('ا' ,'ر') -> Merged: 'را' | Count: 16
```

Figure 11: TOP-10 MOST FREQUENT BPE MERGES

Linguistic Significance:
The Urdu dictionary (`wordlist.pdf`) was parsed using `pypdf` to extract 311 unique word entries as Unicode character sequences after stripping blank lines [1]. Initializing the BPE vocabulary at the character level allowed the algorithm to iteratively learn frequent adjacent character merges across dictionary words [1]. Learned operations were stored as an ordered rule list ($(t_L, t_R) \rightarrow t_{\text{new}}$). Each unsegmented sentence from `50_nospaces.pdf` was initialized as a Unicode sequence and deterministically merged in the exact order of training rule frequencies, segmenting sentences into subwords without runtime search overhead [1,2].

Task P3T2: Average Characters per Word in Urdu Text

```
TASK P3T2 RESULTS
Total Number of Tokens                : 12939
Total Number of Unicode Characters   : 48161
Average Characters per Token          : 3.7222
```

Figure 12: Total number of tokens and average number of characters

Source link(s) of the corpus used: I used these aricles because these were accessible, after trying many other which were causing error because urllib.request was unable to extract the data and were blocked by the sites.

https://www.bbc.com/urdu/articles/cn9w2dyzxl5o
https://www.bbc.com/urdu/articles/cn7l7mjkl45o
https://www.bbc.com/urdu/articles/cpwl12912gpo
https://www.bbc.com/urdu/articles/c3re4r798p5o
https://www.bbc.com/urdu/articles/cn746lljlnjo

Task P3T3: Urdu Text Tokenization and N-Gram Generation

- **Total token count after preprocessing:** 12740
- **Total unique tokens (vocabulary size):** 2178

```
TASK P3T2 RESULTS
Total Number of Tokens                  : 12939
Total Number of Unicode Characters   : 48161
Average Characters per Token            : 3.7222
```

Figure 13: P3T3 stats

Top-20 Trigrams:

```
TOP-20 TRIGRAMS
 1. '('ذریعہ', 'کا', 'تصویر')' : 35
 2. '('محمد', 'شیخ', 'خالد')' : 32
 3. '('سے', 'جانب', 'کی')' : 25
 4. '('سی', 'بی', 'بی')' : 25
 5. '('میں', 'بارے', 'کے')' : 17
 6. '('تھا', 'کہنا', 'کا')' : 15
 7. '('کہ', 'تھا', 'کہنا')' : 15
 8. '('ڈی', 'ایم', 'پی')' : 14
 9. '('سی', 'ڈی', 'ایم')' : 14
10. '('لیے', 'کے', 'کرنے')' : 13
11. '('نے', 'محمد', 'شیخ')' : 11
12. '('کہ', 'ہیں', 'لکھتے')' : 11
13. '('کی', 'اللہ', 'رضا')' : 11
14. '('کہا', 'کہ', 'نے')' : 9
15. '('کا', 'تصویر', 'ہے')' : 9
16. '('اسامہ', 'بن', 'لادن')' : 9
17. '('کے', 'عرب', 'سعودی')' : 9
18. '('تھا', 'گیا', 'کہا')' : 8
19. '('کہ', 'تھا', 'گیا')' : 8
20. '('مطابق', 'کے', 'ان')' : 8
```

Figure 14: Top 20 Trigram

Top-20 Unigrams:

```
TOP-20 UNIGRAMS
 1. '627 : 'کے
 2. '415 : 'میں
 3. '373 : 'کی
 4. '280 : 'اور
 5. '242 : 'سے
 6. '215 : 'کا
 7. '195 : 'نے
 8. '186 : 'پر
 9. '186 : 'کہ
10. '179 : 'ہے
11. '175 : 'کو
12. '163 : 'ہیں
13. '132 : 'اس
14. '112 : 'تھا
15. '96 : 'بھی
16. '91 : 'ایک
17. '90 : 'یہ
18. '85 : 'ان
19. '83 : 'کیا
20. '76 : 'کر
```

Figure 15: Top 20 unigrams

Top-20 Bigrams:

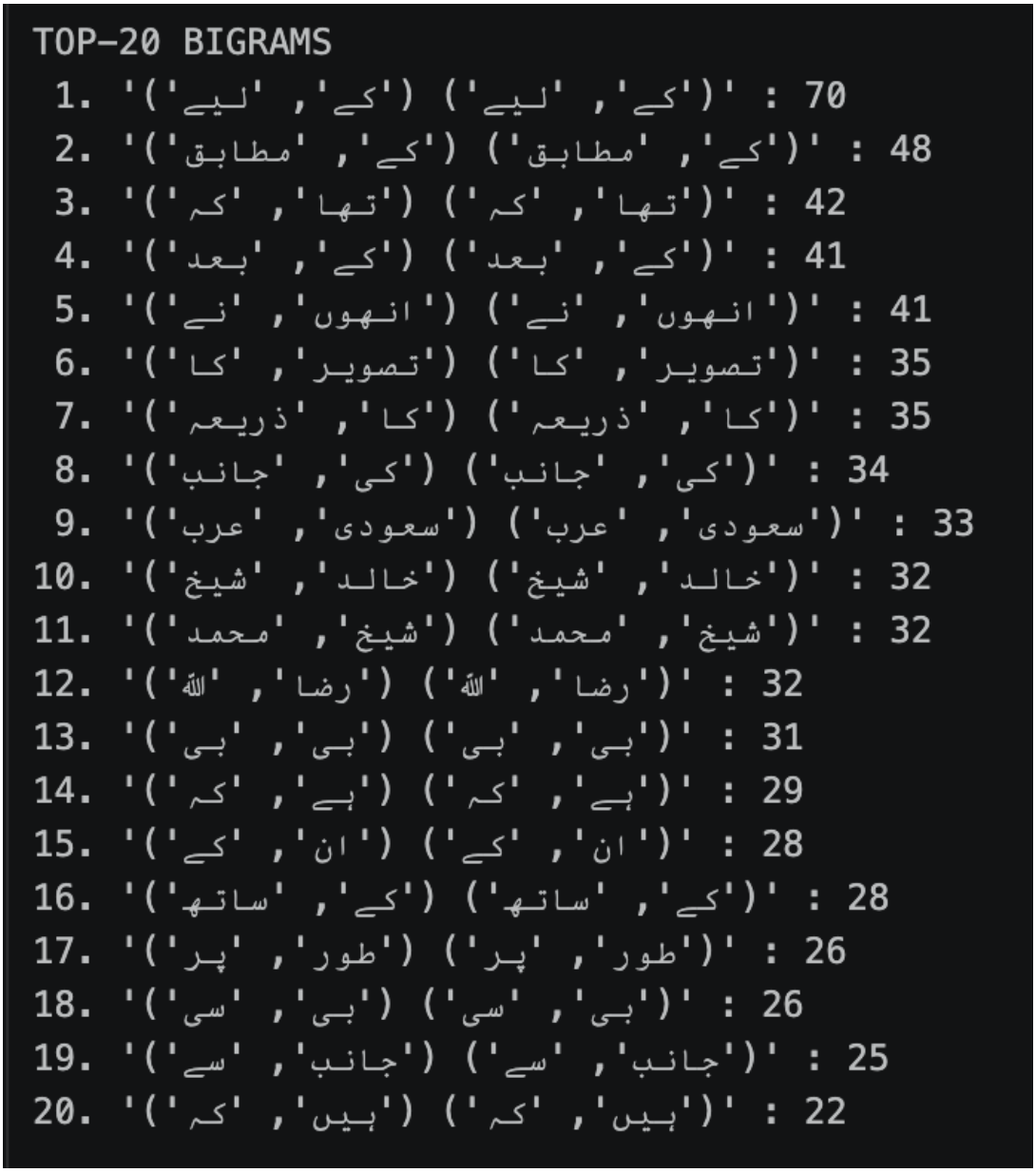


Figure 16: Top 20 bigram

Brief Analysis:

- **Top Unigrams:** kay (627), mai (415), ki (373), aur (280), say (242), ka (215), ne (195), par (186), kya (186), hai (179).
- **Top Bigrams:** ('kay', 'liye') (70), ('kay', 'mutabiq') (48), ('tha', 'kya') (42), ('kay', 'baad') (41), ('unhon', 'ne') (41).
- **Top Trigrams:** ('tasveer', 'ka', 'zariya') (35), ('khalid', 'sheikh', 'mohammad') (32), ('ki', 'janib', 'say') (25), ('b', 'b', 'c') (25), ('kay', 'bary', 'mai') (17).

1. **Grammatical Unigrams:** These are the most frequent words in Urdu grammar which are the basic building blocks of Urdu prose and include function words such as everyday words and postpositions like *kay*, *mai*, *ki* etc.

2. **Meaningful Phrases:** Bigrams and trigrams are used to recall actual journalistic phrases and postpositional phrases, for example *kay liye* (for) or *kay mutabiq* (according to).

3. **Data Sparsity:** Frequent data sparsity is demonstrated by the dramatic drop in frequencies, from unigrams (627) to trigrams (17–35), showing that higher-order models have significant zero-count problems. For this reason, smoothing techniques or BPE (subword tokenization) is necessary.

References

References

[1] Daniel Jurafsky and James H. Martin. (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft). Chapters 2, 3, and 8.

[2] Steven Bird, Ewan Klein, and Edward Loper. (2009). *Natural Language Processing with Python*. O'Reilly Media.

[3] Leonard Richardson. (2026). *Beautiful Soup 4 Documentation*.

[4] Python Software Foundation. (2026). *Python 3 Standard Library: re Module Documentation*.

[5] PyPDF Contributors. (2026). *PyPDF Documentation*.

[6] NLTK Development Team. (2009). *NLTK Corpora (Brown Corpus and Inaugural Address Corpus)*.

[7] BBC Urdu. (2026). *Urdu News Corpus*. Retrieved from `https://www.bbc.com/urdu`.

[8] Regex101. (2026). *Regex101: Online Regex Tester and Debugger*. Retrieved from `https://regex101.com/`